

DEVSECOPS LAB BOOK · PHASE 6: CONTAINER ORCHESTRATION

Chapter 24

Helm

Kubernetes Package Management

Package, template, and deploy applications to Kubernetes.
Manage complex applications with values, templates, and dependencies.

TIME ESTIMATE: 3–4 HOURS

24.0 · CHAPTER GOAL

What You'll Walk Away With

Instructor Framing

Position Helm as the package manager for Kubernetes — the same mental model as apt or npm, but for whole application stacks. Every chart is a reusable, versioned deployment unit that replaces hand-written, duplicated YAML.

MENTAL MODEL

Chart — The package — templates + metadata + defaults

Release — A deployed instance of a chart, tracked by Helm

- Helm chart structure understood
- Package applications into charts
- Deploy with values customization
- Manage releases (install, upgrade, rollback)
- Use public charts (stable applications)

From Repetitive YAML to Reusable Templates

Without Helm

Deployment, Service, and ConfigMap are all hand-written per environment — prod, staging, dev each need their own near-duplicate YAML files with only small value changes (replica count, image tag, hostnames).

```
metadata:
  name: myapp-prod
  labels: {app: myapp, env: production}
spec:
  replicas: 3
---
kind: Service
  name: myapp-prod-service
---
kind: ConfigMap  # repeated per env...
```

With Helm

One chart with templated YAML reads values from a single values.yaml. Swap environments by swapping the values file — the templates never change.

```
# values.yaml
environment: production
replicas: 3
database_host: postgres-prod...

# templates/deployment.yaml
{{ .Values.environment }}
{{ .Values.replicas }}
{{ .Values.database_host }}

→ One chart, infinite configurations
```

Anatomy of a Chart Directory

```
myapp-chart/
├─ Chart.yaml                  Chart metadata
├─ values.yaml                 Default values
├─ templates/                  Kubernetes resources
│   ├─ deployment.yaml
│   ├─ service.yaml
│   ├─ configmap.yaml
│   ├─ secret.yaml
│   └─ _helpers.tpl            Template helpers
├─ charts/                     Subchart dependencies
└─ README.md                   Documentation
```

Chart.yaml

Metadata: name, version, appVersion, maintainers

values.yaml

Default configuration values consumed by templates

templates/

Go-templated Kubernetes manifests rendered at install time

charts/

Vendored or downloaded subchart dependencies

Chart.yaml vs. values.yaml

Chart.yaml

```yaml
apiVersion: v2
name: myapp
description: A Helm chart for MyApp
type: application
version: 1.0.0
appVersion: "2.1.0"
keywords: [myapp, application]
home: https://myapp.dev
maintainers:
  - name: John Doe
```

Identifies the chart: name, version, appVersion, ownership.

values.yaml

```yaml
replicaCount: 3
image:
  repository: harbor.../myapp
  tag: "1.0.0"
service:
  type: LoadBalancer
  port: 80
resources:
  limits: {cpu: 500m, memory: 512Mi}
autoscaling:
  enabled: true
```

The knobs: every value a template can reference & override.

Go Templates Render Kubernetes YAML

templates/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "myapp.fullname" . }}
spec:
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
  {{- end }}
  template:
    spec:
      containers:
      - name: {{ .Chart.Name }}
        image: "{{ .Values.image.repository }}:
{{ .Values.image.tag }}"
        resources:
          {{- toYaml .Values.resources | nindent 12 }}
```

_helpers.tpl

```
{{- define "myapp.fullname" -}}
{{ .Release.Name }}-{{ .Chart.Name }}
{{- end }}
```

Reusable naming & label snippets, included across templates.

READ THE SYNTAX

{{ .Values.x }} — pull from values.yaml

{{ .Release.Name }} — release-scoped name

{{- if }} / {{- end }} — conditionals

nindent 12 — reindent nested YAML

Install · Upgrade · Rollback · Uninstall

Install

```
helm install myapp ./chart --set replicaCount=5
```

Upgrade

```
helm upgrade --atomic myapp ./chart --set
image.tag=2.1.0
```

Rollback

```
helm rollback myapp 2
```

Uninstall

```
helm uninstall myapp -n myapp-production
```

Artifact Hub: Install Stable Applications

Add Repo

```
helm repo add jetstack
https://charts.jetstack.io
```

Update

```
helm repo update
```

Search

```
helm search repo prometheus
```

Install

```
helm install cert-manager
jetstack/cert-manager
```

COMMON REPOSITORIES

prometheus-community	https://prometheus-community.github.io/helm-charts	*Monitoring stack*
jetstack	https://charts.jetstack.io	*cert-manager (HTTPS/TLS)*
bitnami	https://charts.bitnami.com/bitnami	*PostgreSQL, Redis, and more*

Composing Stacks from Subcharts

Chart.yaml

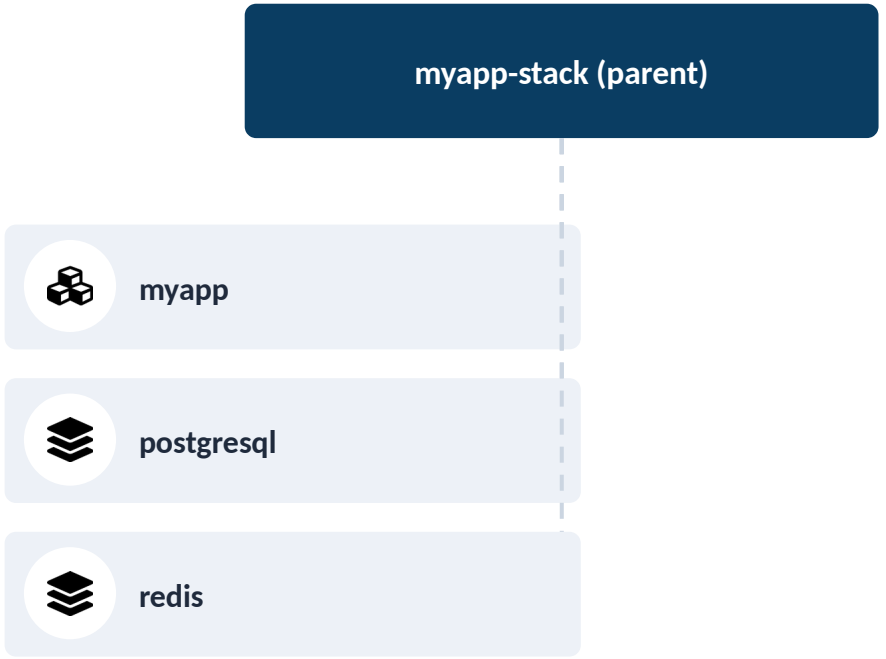
```
name: myapp-stack

dependencies:
  - name: postgresql
    version: "11.x.x"
    repository: https://charts.bitnami.com/bitnami
    condition: postgresql.enabled

  - name: redis
    version: "17.x.x"
    condition: redis.enabled

  - name: myapp
    repository: file://../myapp-chart
```

Composed Stack



`helm dependency update`

downloads charts into charts/

One Chart, Many Environments

values-prod.yaml

replicaCount	5
image.tag	2.1.0
resources.requests.cpu	1000m
autoscaling.minReplicas	5
autoscaling.maxReplicas	20
postgresql.persistence	100Gi

values-dev.yaml

replicaCount	1
image.tag	latest
resources.requests.cpu	100m
autoscaling.enabled	false
autoscaling.maxReplicas	—
postgresql.persistence	10Gi

```
helm install myapp ./chart -f values-prod.yaml -n production   |   helm install myapp-dev ./chart -f values-dev.yaml -n development
```

Preview Before You Deploy

Preview Changes

```
helm install myapp ./chart \
  --dry-run --debug

helm template myapp ./chart
```

Renders manifests locally without touching the cluster — the fastest way to catch templating mistakes.

Lint the Chart

```
helm lint ./myapp-chart
```

```
[INFO]        Chart.yaml: icon is missing

[WARNING]     Chart.yaml: unsupported API version

[ERROR]       values.yaml: invalid YAML
```

Catches structural and syntax errors before install/upgrade.

Feature Reference

Feature	Purpose	Example
Chart.yaml	Metadata	`Name, version, description`
values.yaml	Default config	`Replicas, image, resources`
Templates	Kubernetes YAML	`Deployment, Service`
Helm Install	Deploy application	`helm install myapp ./chart`
Helm Upgrade	Update version	`helm upgrade myapp ./chart`
Helm Rollback	Revert to previous	`helm rollback myapp`

Chart · Release · Repository — the three nouns that make Helm click.



KEY TAKEAWAY

Reusable, Manageable Applications

BEFORE HELM

- ✕ Complex YAML files
- ✕ Repetitive configuration
- ✕ Hard to share applications
- ✕ Manual version management

AFTER HELM

- ✓ Templated, DRY YAML
- ✓ Configurable with values
- ✓ Share charts on Artifact Hub
- ✓ One-command deployments